

EXAMEN ORDER

Aparición de las 4 cajas en el header



Crea un `renderAnalytics` donde mete las 4 cajas dentro de un `view`

```
const renderAnalytics = () => {
  return (
    <View style={styles.analyticsContainer}>
      <View style={styles.analyticsRow}>
        <View style={styles.analyticsCell}>
          <TextRegular textStyle={styles.text}>
            Invoiced today
          </TextRegular>
          <TextSemiBold textStyle={styles.text}>
            {analytics.invoicedToday.toFixed(2)}€
          </TextSemiBold>
        </View>
        <View style={styles.analyticsCell}>
          <TextRegular textStyle={styles.text}>
            #Pending orders
          </TextRegular>
          <TextSemiBold textStyle={styles.text}>
            {analytics.numPendingOrders}
          </TextSemiBold>
        </View>
      </View>
      <View style={styles.analyticsRow}>
        <View style={styles.analyticsCell}>
          <TextRegular textStyle={styles.text}>
            #Delivered today
          </TextRegular>
          <TextSemiBold textStyle={styles.text}>
            {analytics.numDeliveredTodayOrders}
          </TextSemiBold>
        </View>
        <View style={styles.analyticsCell}>
          <TextRegular textStyle={styles.text}>
            #Yesterday orders
          </TextRegular>
          <TextSemiBold textStyle={styles.text}>
            {analytics.numYesterdayOrders}
          </TextSemiBold>
        </View>
      </View>
    </View>
  )
}
```

Dentro del `view` gigante, hay dos `views` hijos (parte izquierda, parte derecha). Y dentro de estos `views` hijos hay otros dos `views` donde diferencia cada cuadrado.

Para conseguir que los cuadrados se agrupen de esa forma hay que hacerlo de la siguiente manera en el `styles`:

```
analyticsContainer: {
  backgroundColor: GlobalStyles.brandPrimaryTap,
  paddingVertical: 10
},
analyticsRow: {
  flexDirection: 'row', // SOLUTION. Exercise - Restaurant analytics
  justifyContent: 'space-around' // SOLUTION. Exercise - Restaurant analytics
},
```

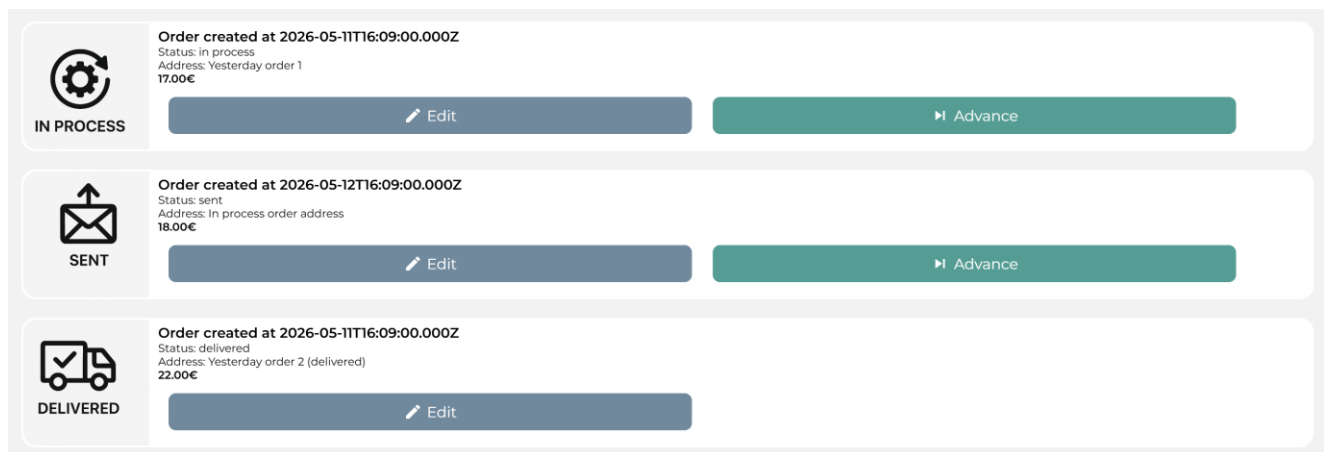
Apila los `views` hijos en fila con el `flexDirection`, y con poco espacio entre ellos. Al ser dos `views` distintos, primero se dibujan los dos primeros (Invoiced today y Pending orders) y abajo se dibujan los otros dos (Delivered today y yesterday orders)

Se implementa así en el renderHeader:

```
const renderHeader = () => {
  return (
    <View>
      <ImageBackground source={({restaurant?.heroImage}) ? { uri: API_BASE_URL + '/' + restaurant.heroImage, cache: 'force-cache' } : null} style={styles.image} />
      <View style={styles.restaurantHeaderContainer}>
        <TextSemiBold textStyle={styles.textTitle}><{restaurant.name}</TextSemiBold>
        <Image style={styles.image} source={restaurant.logo ? { uri: API_BASE_URL + '/' + restaurant.logo, cache: 'force-cache' } : null} />
        <TextRegular textStyle={styles.description}><{restaurant.description}</TextRegular>
        <TextRegular textStyle={styles.description}><{restaurant.restaurantCategory ? restaurant.restaurantCategory.name : ''}</TextRegular>
      </View>
    </ImageBackground>
    </* SOLUTION. Exercice - Restaurant analytics */>
    <{analytics !== null && renderAnalytics()}>
  </View>
)
}
```

Donde lo que hace ahí es preguntar si tiene analytics y si las tiene (analytics no es null por tanto se cumple la condición) dibuja el renderAnalytics dentro de la cabecera

Funcionalidad del botón de avance



Para conseguir que el botón advance se esconda cuando llegue al último status, se implementa así:

```
/* SOLUTION. Exercice - Order status change */
{item.status !== 'delivered' &&
  <Pressable
    onPress={() => handleNextStatus(item)}
    style={({pressed}) => [
      {
        backgroundColor: pressed
          ? GlobalStyles.brandGreenTap
          : GlobalStyles.brandGreen
      },
      styles.actionButton
    ]}>
    <View style={{flex: 1, flexDirection: 'row', justifyContent: 'center'}}>
      <MaterialCommunityIcons name='skip-next' color='white' size={20}/>
      <TextRegular textStyle={styles.text}>Advance</TextRegular>
    </View>
  </Pressable>
}
```

Donde realiza una condición (primera línea) y si se cumple realiza lo primero y si no se realiza lo segundo

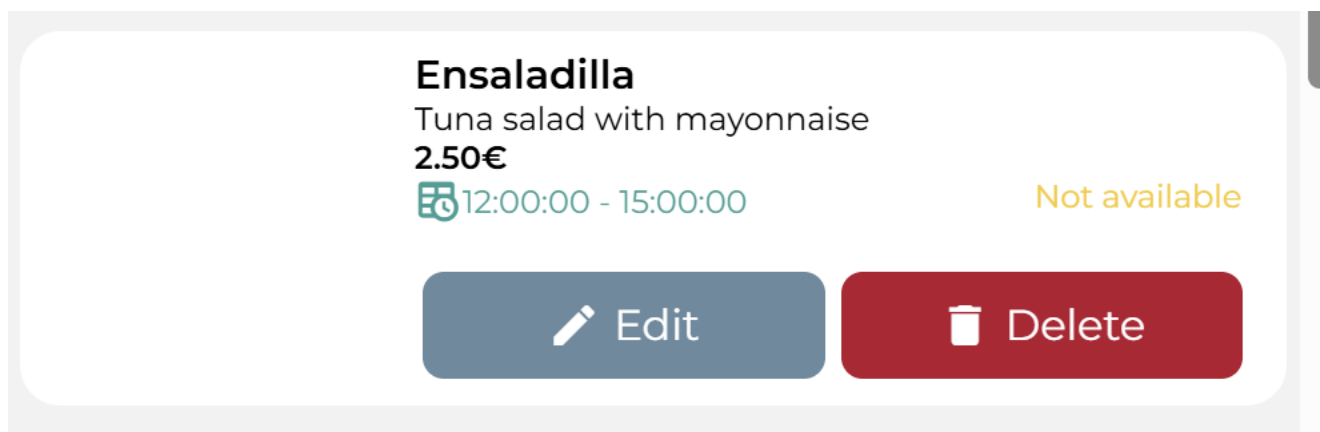
Para que el cambio de estado se produzca se hace la siguiente función

```
const handleNextStatus = async (order) => {  
  /// SOLUTION. Excercise - Order status change  
  try {  
    await nextStatus(order)  
    showMessage({  
      message: `Order ${order.id} status updated`,  
      type: 'success',  
      style: GlobalStyles.flashStyle,  
      titleStyle: GlobalStyles.flashTextStyle  
    })  
    fetchRestaurantOrders()  
    fetchRestaurantAnalytics()  
  } catch (error) {  
    showMessage({  
      message: `Error advancing order status. ${error}`,  
      type: 'danger',  
      style: GlobalStyles.flashStyle,  
      titleStyle: GlobalStyles.flashTextStyle  
    })  
  }  
}
```

Donde se llama al fetchRestaurantOrders y al fetchRestaurantAnalytics para que se aincronice la pantalla con el estado del order en la base de datos.

EXAMEN SCHEDULES

Cómo conseguir los textos separados y de distinto color



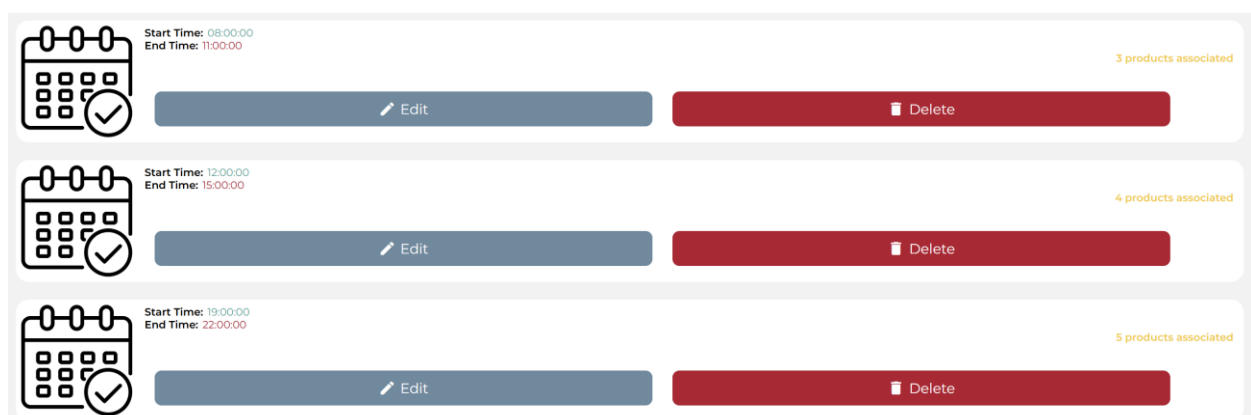
Para conseguir que esos dos textos aparezcan separados y con colores diferentes hay que hacer un View. Este View le pondremos `flexDirection: 'row'` y `justifyContent: 'space-between'`. Con esto conseguiremos que aparezcan en línea.

Para este en concreto hay que tener en cuenta que el icono del reloj y las horas son objetos distintos, por tanto, hay que meterlos en un View para que estén agrupados a la izquierda. Si no lo agrupamos en el view, el view padre los tomará como dos objetos distintos y los separará por culpa del `justifyContent`.

Además, en este caso, si no tiene horario, habría que dibujar “Not Schedule” y si está disponible no habría que dibujar nada. Lo podemos ver en este código implementado:

```
<View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
  <View style={{ flexDirection: 'row', alignItems: 'center' }}>
    {item.schedule
      ? <>
        <MaterialCommunityIcons name="timetable" size={18} color={GlobalStyles.brandGreen} />
        <TextRegular textStyle={{ color: GlobalStyles.brandGreen }}>{item.schedule.startTime} - {item.schedule.endTime}</TextRegular>
      </>
      : <>
        <MaterialCommunityIcons name="timetable" size={18} color={GlobalStyles.brandPrimary} />
        <TextRegular textStyle={{ color: GlobalStyles.brandPrimary }}>Not scheduled</TextRegular>
      </>
    }
  </View>
  <View>
    {!item.availability &&
      <TextRegular textStyle={styles.availability}>Not available</TextRegular>
    }
  </View>
</View>
```

Conseguir textos de diferentes colores



Para conseguir los textos de diferentes colores como se ve en la imagen, lo único que hay que hacer es crear un View padre y dentro hacer Views hijos para cada línea con `flexDirection: 'row'` y `justifyContent= 'flex-start'` y dentro poner los diferentes textos

Para conseguir los productos asociados a la derecha, habría que crear un view hijoy ponerle el `justifyContent= 'flex-end'`.

Quedaría así:

```
<View>
  <View style={{ flex: 1, flexDirection: 'row' }}>
    <TextSemiBold>Start Time:</TextSemiBold>
    <TextRegular textStyle={{ marginLeft: 5, color: GlobalStyles.brandGreen }}>{item.startTime}</TextRegular>
  </View>
  <View style={{ flex: 1, flexDirection: 'row' }}>
    <TextSemiBold>End Time:</TextSemiBold>
    <TextRegular textStyle={{ marginLeft: 5, color: GlobalStyles.brandPrimary }}>{item.endTime}</TextRegular>
  </View>
  <View style={{ flex: 1, flexDirection: 'row', justifyContent: 'flex-end' }}>
    <TextSemiBold textStyle={{ color: item.products.length === 0 ? GlobalStyles.brandPrimary : GlobalStyles.brandSecondary }}>{item.products.length} products associated</TextSemiBold>
  </View>
</View>
```

Restricción personalizada

Start Time (HH:mm:ss):

End Time (HH:mm:ss):

The time must be in the HH:mm (e.g. 14:30:00) format

Para conseguir esa restricción habría que hacer lo siguiente.

```
const validationSchema = yup.object().shape({
  startTime: yup
    .string()
    .required('Start time is required')
    .matches(
      /^[01]\d|2[0-3]:([0-5]\d):([0-5]\d)$/,
      'The time must be in the HH:mm (e.g. 14:30:00) format'
    ),
  endTime: yup
    .string()
    .required('End time is required')
    .matches(
      /^[01]\d|2[0-3]:([0-5]\d):([0-5]\d)$/,
      'The time must be in the HH:mm (e.g. 14:30:00) format'
    )
})
```

La función `matches` significa que debe coincidir con ese patrón.

^ y \$: Significan que el texto debe empezar y terminar exactamente así. No se permite escribir "A las 14:30:00 horas" (tiene que ser solo el número).

([01]\d|2[0-3]) (Las Horas): Obliga a que los dos primeros números sean desde **00** hasta **19**, ó (|) desde **20** hasta **23**. (Si pones un 24 o un 99, da error).

([0-5]\d) (Los Minutos y los segundos): Obliga a que los siguientes dos números vayan desde **00** hasta **59**

\d significa cualquier número del 0 al 9